Case Study Report on

ACTIVITY NAVIGATION APP

project submitted in fulfilment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY IN CSE (AI & ML)

Submitted by

M. Deekshitha	**A23126552217**
M. Surya Karthik	**A23126552218**
M. Triveni	**A23126552219**
M. Y V G Karthikeya	**A23126552220**
M. Mohith Kumar	**A23126552221**
M. Karthikeya	**A23126552222**
N. Sri Chandu	**A23126552223**
N. Abhiram	**A23126552224**
N. Sai Poojitha	**A23126552225**
P. Dayananda Sagar	**A23126552226**

Under the Guidance of

Mr. P. Santosh Kumar

(Assistant Professor)

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING (AI & ML)

ANIL NEERUKONDA INSTITUTE OF TECHNOLOGY AND SCIENCES

(UGC AUTONOMOUS)

(Permanently Affiliated to AU, Approved by AICTE and Accredited by NBA

& NAAC with 'A+' Grade)

SANGIVALA, BHEEMILI MANDAL, VISAKHAPATNAM – 531162

2025 – 2026

ANIL NEERUKONDA INSTITUTE OF TECHNOLOGY AND SCIENCES

(UGC AUTONOMOUS)

(Permanently Affiliated to AU, Approved by AICTE and Accredited by NBA & NAAC with 'A+' Grade)

SANGIVALA, BHEEMILI MANDAL, VISAKHAPATNAM – 531162

BONAFIDE CERTIFICATE

This is to certify that the case study entitled "**Activity Navigation"** submitted by **M DEEKSHITHA, M SURYA KARTHIK, M TRIVENI, M Y V G KARTHIKEYA , M MOHITH KUMAR, M KARTHIKEYA, N SRI CHANDU, N ABHI RAM, N SAI POOJITHA , P DAYANANDA SAGAR** in fulfillment of the requirements for the award of Bachelor of Technology in CSE (AI & ML) in the department of Computer Science and Engineering (AI & ML) of Anil Neerukonda Institute of technology and sciences (A+), Visakhapatnam is a record of Bonafide work carried out under my guidance and supervision.

Mr. P. Santosh Kumar

ASSISTANT PROFESSOR

Department of CSE (AI & ML)

ACKNOWLEDGEMENT

This case study reflects our academic learning and professional development journey. The case study provided an opportunity to apply classroom knowledge in a real-world cloud and containerization environment.

We express sincere gratitude to Dr. K. Selvani Deepthi, Head of the Department of Computer Science and Engineering(AI&ML) at ANITS, and our guide Mr. Santosh Kumar P, Associate Professor, for invaluable guidance and encouragement throughout the case study. We also acknowledge the faculty members for their unwavering support.

M. Deekshitha	**A23126552217**
M. Surya Karthik	**A23126552218**
M. Triveni	**A23126552219**
M. Y V G Karthikeya	**A23126552220**
M. Mohith Kumar	**A23126552221**
M. Karthikeya	**A23126552222**
N. Sri Chandu	**A23126552223**
N. Abhiram	**A23126552224**
N. Sai Poojitha	**A23126552225**
P. Dayananda Sagar	**A23126552226**

SI.NO.	Title	Page No.
1	**Abstract**	**5**
2	**Introduction**	**5**
3	**Aim**	**6**
4	**Objective**	**6**
5	**Tools Used**	**6**
6	**Architecture**	**7**
7	**Methodology**	**8**
8	**Procedure**	**10**
9	**Implementation**	**10**
10	**Output**	**14**
11	**Conclusion**	**15**
12	**Advantages**	**16**
13	**Limitations**	**16**
14	**References**	**16**

Abstract

The Activity Navigation Application is a simple Android-based mobile application developed to demonstrate the concept of navigation between different screens using Explicit Intents. The primary objective of this project is to provide a clear understanding of how activities interact and transition within an Android application.

The application consists of two main screens: MainActivity and SecondActivity. The MainActivity acts as the entry point, allowing users to navigate to the second screen through a button click. This navigation is implemented using Explicit Intents, ensuring a direct and controlled transition between activities. The SecondActivity provides an option to return to the previous screen using a “Go Back” button, demonstrating the use of the activity lifecycle and the finish() method.

The project is developed using Kotlin programming language and XML layouts for designing a simple and user-friendly interface. Additionally, Toast messages are incorporated to provide feedback to users during navigation. The application focuses purely on front-end functionality without involving backend processing or database integration.

Overall, this project serves as a foundational example for beginners to understand Android activity navigation, user interaction, and the integration of core components in mobile application development.

Introduction

The Activity Navigation Application is a simple Android-based mobile application developed to demonstrate the concept of navigation between different screens using Explicit Intents. In modern mobile applications, navigation plays a crucial role in providing a smooth and interactive user experience. This project focuses on implementing basic navigation functionality in a clear and structured way.

The application consists of two activities: MainActivity and SecondActivity. The MainActivity serves as the entry point of the application, where the user is presented with a button to navigate to the second screen. When the user clicks the button, an Explicit Intent is used to open the SecondActivity. The SecondActivity provides a "Go Back" button, which allows the user to return to the MainActivity using the finish() method.

The application is developed using Kotlin programming language and XML layouts for designing the user interface. It also includes Toast messages to provide feedback to the user during navigation. This project mainly focuses on front-end interaction and does not involve backend processing or database integration.

Overall, the project provides a strong foundation for understanding Android activity navigation and helps beginners learn how different components of an Android application work together.

Aim

The aim of this project is to design and develop a simple Android application that demonstrates navigation between multiple activities using Explicit Intents. It focuses on understanding how different screens (activities) interact, how user actions trigger navigation, and how basic Android components such as buttons, Toast messages, and the activity lifecycle work together to provide a smooth user experience.

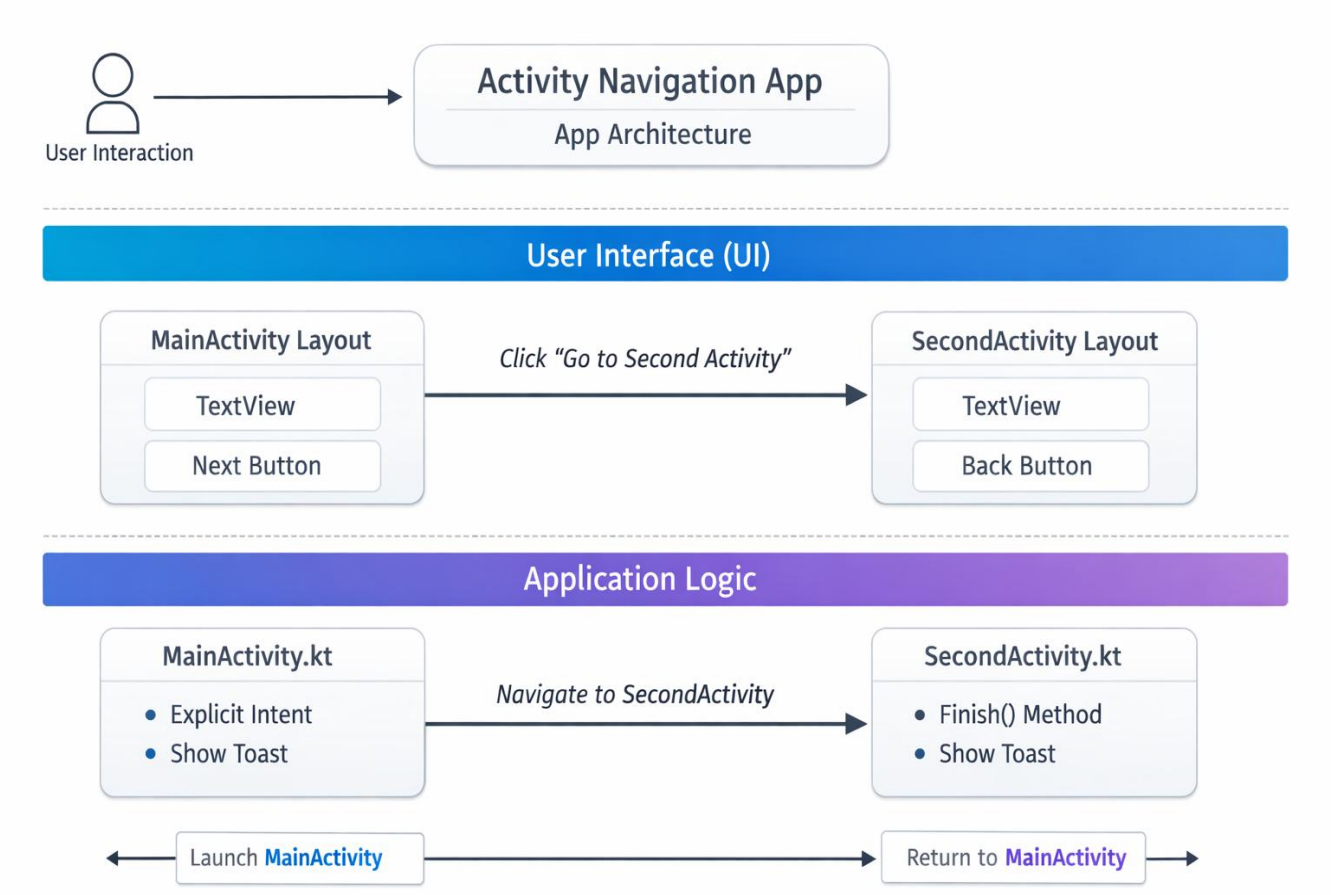
Objectives

- To understand the concept of activity navigation in Android applications using Explicit Intents.
- To design and develop a simple user interface using XML layouts.
- To implement button click events for user interaction.
- To demonstrate navigation between multiple screens (MainActivity and SecondActivity).
- To use Toast messages to provide feedback during user actions.
- To understand the basic activity lifecycle and the use of the finish() method for returning to the previous screen.

Tools & Libraries

- **Programming Language:** Kotlin
- **Frontend Design:** XML Layouts
- **Framework:** Android SDK
- **Libraries/Concepts Used:**
    - Explicit Intent
    - Activity Lifecycle
    - Toast Messages
    - Button Click Listeners
    - LinearLayout UI Design

Architecture



Methodology

The development of the Activity Navigation Application was carried out using the following step-by-step procedure:

1. Opened Android Studio and created a new project.
2. Selected the *Empty Views Activity* template.
3. Configured the project by choosing Kotlin as the programming language and setting the minimum SDK version.
4. Designed the user interface for MainActivity using *activity_main.xml* with a TextView and a Button.
5. Created a new layout file *activity_second.xml* for SecondActivity with a TextView and a "Go Back" button.
6. Applied proper layout properties such as alignment, padding, and background to ensure a clean and user-friendly interface.
7. Created *MainActivity.kt* and linked it with the XML layout using `setContentView()`.
8. Connected UI components using `findViewById()` and declared required variables.
9. Implemented a click listener for the button in MainActivity using Kotlin lambda function.
10. Displayed a Toast message when the button is clicked to provide user feedback.
11. Used Explicit Intent (`Intent(this, SecondActivity::class.java)`) to navigate to SecondActivity.
12. Created *SecondActivity.kt* and linked it with *activity_second.xml*.
13. Added a click listener for the "Go Back" button in SecondActivity.
14. Displayed a Toast message before returning to the previous screen.
15. Used the `finish()` method to close SecondActivity and return to MainActivity.
16. Updated the *AndroidManifest.xml* file to declare both activities and set MainActivity as the launcher activity.
17. Built and compiled the project using Android SDK tools.
18. Ran the application on an emulator or physical Android device.
19. Tested navigation, button functionality, and Toast messages for correct behavior.
20. Verified that the application runs smoothly with proper user interaction.

Project Folder Structure

The ActivityNavigationApp follows the standard Android project structure organized by Android Studio.

```
ActivityNavigationApp/
│
├── app/
│   ├── src/
│   │   ├── main/
│   │   │   ├── java/com/example/activitynavigation/
│   │   │   │   ├── MainActivity.kt
│   │   │   │   └── SecondActivity.kt
│   │   │   │
│   │   │   ├── res/
│   │   │   │   ├── layout/
│   │   │   │   │   ├── activity_main.xml
│   │   │   │   │   └── activity_second.xml
│   │   │   │
│   │   │   └── AndroidManifest.xml
│   │
│   └── build.gradle
│
├── gradle/
├── build.gradle (Project level)
└── settings.gradle
├── settings.gradle
└── gradlew / gradlew.bat
```

PROCEDURE

- Install and open Android Studio on your system.
- Create a new Android project using Kotlin as the programming language.
- Select an Empty Activity template and configure project details (name, package, minimum SDK).
- Design the user interface for MainActivity using XML in the activity_main.xml file.
- Add UI components such as TextView and Button to the layout.
- Set the title and description text for the MainActivity screen.
- Create a second layout file activity_second.xml for SecondActivity.
- Add TextView and a “Go Back” button in the SecondActivity layout.
- Open MainActivity.kt and connect UI components using findViewById().
- Implement a click listener for the button in MainActivity.
- Display a Toast message when the button is clicked.
- Use an Explicit Intent to navigate from MainActivity to SecondActivity.
- Open SecondActivity.kt and connect the “Go Back” button.
- Implement a click listener in SecondActivity to handle the back action.
- Display a Toast message before returning to the previous screen.
- Use the finish() method to close SecondActivity and return to MainActivity.
- Update the AndroidManifest.xml file to declare both activities.
- Run the application on an emulator or physical Android device.
- Test navigation, button functionality, and Toast messages for correct behavior.

Implementation (Code)

```
// MainActivity.kt

package com.example.activitynavigation

// Import required Android libraries
import android.content.Intent        // For creating and launching Intents
import android.os.Bundle             // For the onCreate lifecycle method
import android.widget.Button         // For the Button widget
import android.widget.Toast          // For showing short popup messages
import androidx.appcompat.app.AppCompatActivity // Base class for all activities

class MainActivity : AppCompatActivity() {

    private lateinit var btnGoToSecond: Button

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)

        setContentView(R.layout.activity_main)

        btnGoToSecond = findViewById(R.id.btnGoToSecond)

        btnGoToSecond.setOnClickListener {

            Toast.makeText(
                this,                    // Context (current activity)
                "Opening Second Activity!", // Message to display
```

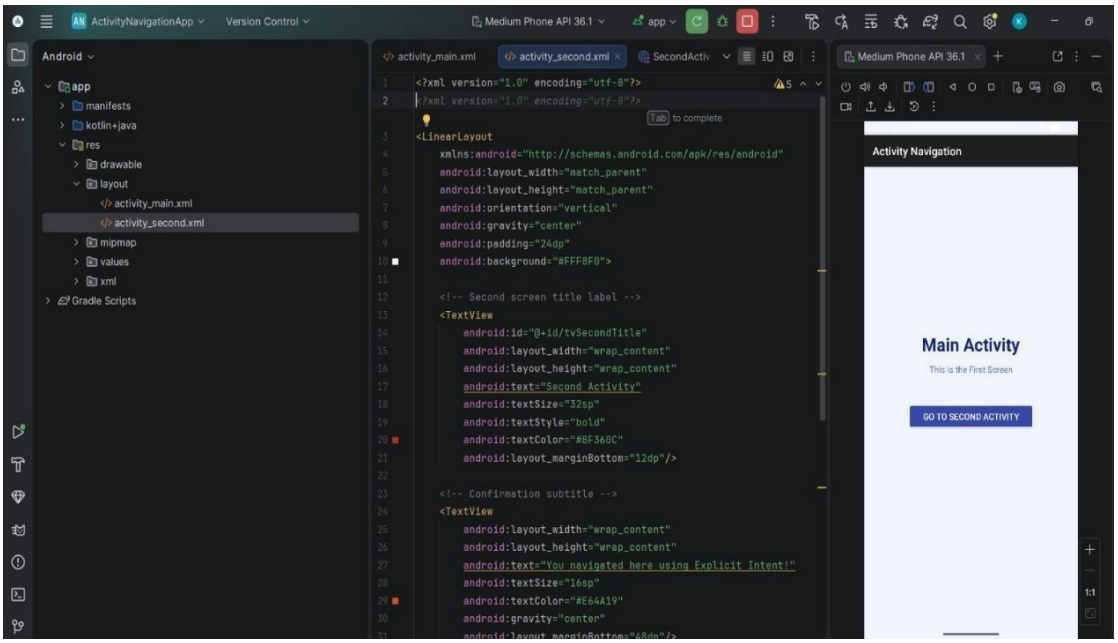
```
            Toast.LENGTH_SHORT          // Duration
        ).show()

        val intent = Intent(this, SecondActivity::class.java)
        startActivity(intent)
    }
  }
}


//  SecondActivity.kt

package com.example.activitynavigation

// Import required Android libraries
import android.os.Bundle            // For the onCreate lifecycle method
import android.widget.Button        // For the Button widget
import android.widget.Toast         // For showing short popup messages
import androidx.appcompat.app.AppCompatActivity // Base class for all activities

class SecondActivity : AppCompatActivity() {

  private lateinit var btnGoBack: Button

  override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)

    setContentView(R.layout.activity_second)

    btnGoBack = findViewById(R.id.btnGoBack)

    btnGoBack.setOnClickListener {

      Toast.makeText(
        this,                       // Context
        "Going back to Main Activity!", // Message
        Toast.LENGTH_SHORT              // Duration
      ).show()

      finish()
    }
  }
}
```

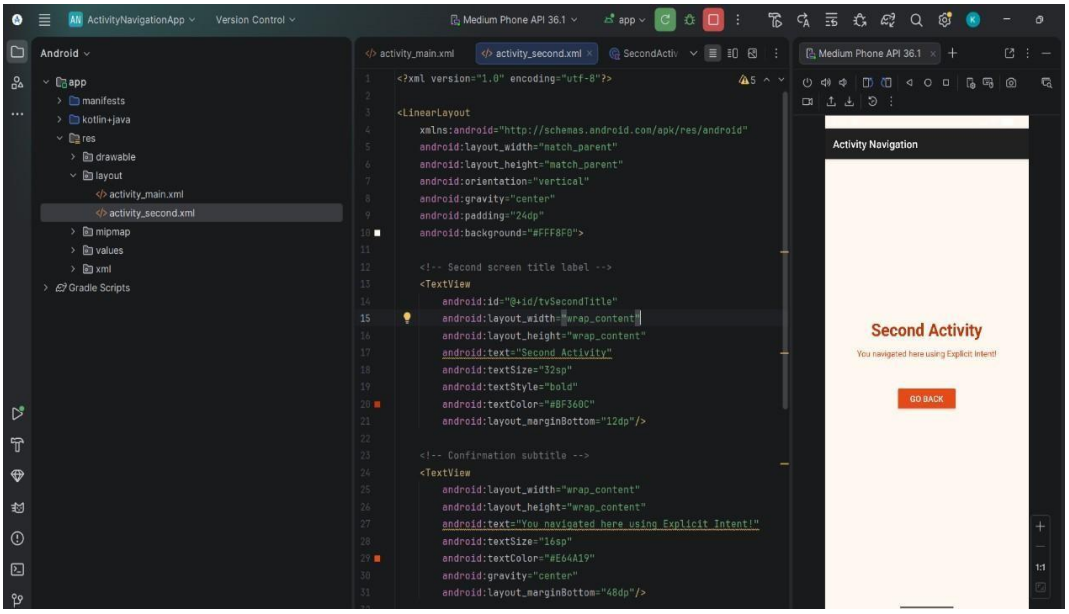
XML Layout (activity_main.xml)

```
<?xml version="1.0" encoding="utf-8"?>
<!--
    LinearLayout: Stacks children vertically, one below the other.
    gravity="center" → centers all children horizontally and vertically.
    background: soft blue tone for the Main screen.
-->
<LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:gravity="center"
    android:padding="24dp"
    android:background="#F0F4FF">

    <!-- Main title label -->
    <TextView
        android:id="@+id/tvMainTitle"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Main Activity"
        android:textSize="32sp"
        android:textStyle="bold"
        android:textColor="#1A237E"
        android:layout_marginBottom="12dp"/>

    <!-- Supporting subtitle text -->
    <TextView
        android:layout_width="wrap_content"
```

```
        android:layout_height="wrap_content"
        android:text="This is the First Screen"
        android:textSize="16sp"
        android:textColor="#5C6BC0"
        android:layout_marginBottom="48dp"/>

    <!-- Navigation button — triggers Explicit Intent in MainActivity.kt -->
    <Button
        android:id="@+id/btnGoToSecond"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Go to Second Activity"
        android:textSize="16sp"
        android:paddingStart="28dp"
        android:paddingEnd="28dp"
        android:paddingTop="12dp"
        android:paddingBottom="12dp"
        android:backgroundTint="#3949AB"
        android:textColor="#FFFFFF"/>
</LinearLayout>
```

Output Screenshots:





Evaluation

Manual Evaluation:

- Verified navigation between activities
- Checked button functionality
- Observed Toast messages

Observation:

- Application runs smoothly without errors
- User interaction is simple and effective
- Navigation flow is clear and understandable

RESULTS

The Activity Navigation Application was successfully developed and executed. The application performs all intended functionalities effectively and provides a smooth user experience.

• The application launches with the MainActivity as the initial screen, displaying a clear and user-friendly interface.
• When the user clicks the "Go to Second Activity" button, the application navigates successfully to SecondActivity using an Explicit Intent.
• Toast messages are displayed during navigation, providing immediate feedback to the user.
• The "Go Back" button in SecondActivity allows users to return to MainActivity using the finish() method.
• Navigation between activities is smooth, responsive, and error-free.
• All UI elements such as buttons and text views function correctly as expected.

Conclusion

The Activity Navigation Application successfully demonstrates the use of Explicit Intents in Android development. The main objective of the project was to understand how to navigate between different activities and manage user interaction effectively.

Through this project, important Android concepts such as activity lifecycle, intent-based navigation, and event handling were implemented. The use of Kotlin and XML helped in building a simple yet efficient mobile application with a clean user interface.

The application provides a practical understanding of real-time navigation features used in modern mobile apps. This project serves as a strong foundation for developing more advanced Android applications involving multiple screens and complex navigation systems.

ADVANTAGES

• Easy to understand and ideal for beginners in Android development

• Clearly demonstrates navigation between activities using Explicit Intents

• Helps in learning core concepts like Activity Lifecycle and event handling

• Provides smooth and responsive navigation experience

• Uses Toast messages to give instant user feedback

• Lightweight application with quick execution

• Can be easily extended by adding more screens and features

LIMITATIONS

• Limited functionality with only two activities

• No database or backend integration

• Does not include advanced UI elements or animations

• No support for user login or personalization

• Static content with no dynamic data updates

• Not suitable for complex or large-scale applications

• Basic error handling with minimal validation

• Focused only on navigation, lacks additional features

Reference: https://github.com/myvgkarthikeya/ActivityNavigationApp.git

Android Developers. *Android Developer Documentation*. Available: https://developer.android.com

JetBrains. *Kotlin Programming Language Documentation*. Available: https://kotlinlang.org

Google. *Material Design Guidelines*. Available: https://material.io

Android Developers. *Activities Overview*. Available: https://developer.android.com/guide/components/activities/intro-activities

Android Developers. *Intents and Intent Filters*. Available: https://developer.android.com/guide/components/intents-filters

Android Developers. *Activity Lifecycle Documentation*. Available: https://developer.android.com/guide/components/activities/activity-lifecycle